

Lab Assignment 11.1

Name: K.SAI VARDHAN REDDY

Roll No: 2503A51266

Identified Errors in Original Document

The following structural and formatting errors were identified in the preliminary draft and corrected for this final submission:

- **Task 1 (Stack):** Corrected line breaks for method calls and return statements.
- **Task 2 (Queue):** Resolved missing vertical spacing between class instantiation and operations.
- **Task 3 (Singly Linked List):** Separated concatenated method calls to improve readability.
- **Task 4 (BST):** Applied proper indentation to the recursive helper function `traverse(node)`.
- **Task 5 (Hash Table):** Separated logic blocks where return statements and driver code were merged.
- **Task 6 (Graph):** Cleaned up vertex and edge definitions with appropriate spacing.
- **Task 7 (Priority Queue):** Extracted driver code from the `display()` method scope.
- **Task 8 (DequeDS):** Corrected concatenation between class instantiation and method returns.
- **Tasks 9 & 10:** Structured mapping tables with clear headers and isolated code implementation blocks.

Task 1: Stack Implementation

Question: Use AI to generate a Stack class with `push`, `pop`, `peek`, and `is_empty` methods.

Prompt used for AI: Generate a Python Stack class using a list. Include `push`, `pop`, `peek`, and `is_empty` methods. Add clear docstrings, handle `pop/peek` on

an empty stack safely, and provide a short example that demonstrates all four operations.

Answer / Code:class Stack:

```
    """A simple LIFO stack implemented using a Python list."""
```

```
    def __init__(self):
```

```
        self.items = []
```

```
    def push(self, item):
```

```
        """Add an item to the top of the stack."""
```

```
        self.items.append(item)
```

```
    def pop(self):
```

```
        """Remove and return the top item, or return None if empty."""
```

```
        if self.is_empty():
```

```
            return None
```

```
        return self.items.pop()
```

```
    def peek(self):
```

```
        """Return the top item without removing it, or None if empty."""
```

```
        if self.is_empty():
```

```
            return None
```

```
        return self.items[-1]
```

```
    def is_empty(self):
```

```

        """Return True when the stack contains no items."""

        return len(self.items) == 0

stack = Stack()

stack.push(10)

stack.push(20)

print("Top:", stack.peek())

print("Popped:", stack.pop())

print("Empty:", stack.is_empty())

```

Explanation: The stack follows LIFO (Last In, First Out). The list's `append` and `pop` operations efficiently manage the top of the stack.

Task 2: Queue Implementation

Question: Use AI to implement a Queue using Python lists.

Prompt used for AI:Generate a Python Queue class using a list. Include enqueue, dequeue, peek, and size methods. Use FIFO behavior, handle an empty queue safely, add docstrings, and include a small demonstration.

Answer / Code:

class Queue:

"""A simple FIFO queue implemented using a Python list."""

def init(self):

self.items = []
def enqueue(self, item):

"""Add an item to the rear of the queue."""

self.items.append(item)

def dequeue(self):

```
"""Remove and return the front item, or return None if empty."""
```

```
if not self.items:
```

```
    return None
```

```
    return self.items.pop(0)
```

```
def peek(self):
```

```
    """Return the front item without removing it, or None if empty."""
```

```
    if not self.items:
```

```
        return None
```

```
    return self.items[0]
```

```
def size(self):
```

```
    """Return the number of items in the queue."""
```

```
    return len(self.items)
```

```
queue = Queue()
```

```
queue.enqueue("Student 1")
```

```
queue.enqueue("Student 2")
```

```
print("Front:", queue.peek())
```

```
print("Removed:", queue.dequeue())
```

```
print("Size:", queue.size())
```

****Explanation:**** This queue follows FIFO (First In, First Out) logic; the element that has been in the list the longest is the first one to be removed.

Task 3: Singly Linked List

****Question:**** Use AI to generate a Singly Linked List with `insert` and `display` methods.

****Prompt used for AI:****

> Generate a Python singly linked list with a Node class and LinkedList class. Implement insert to add nodes at the end and display to print the list. Add docstrings and demonstrate the result.

****Answer / Code:****

```
```python
```

```
class Node:
```

```
 """A node containing data and a reference to the next node."""
```

```
 def __init__(self, data):
```

```
 self.data = data
```

```
 self.next = None
```

```
class LinkedList:
```

```
 """A singly linked list."""
```

```
 def __init__(self):
```

```
 self.head = None
```

```
 def insert(self, data):
```

```
 """Insert a new node at the end of the list."""
```

```
 new_node = Node(data)
```

```
 if self.head is None:
```

```
 self.head = new_node
```

```
 return
```

```

current = self.head

while current.next:

 current = current.next

current.next = new_node

def display(self):

 """Display all values in the linked list."""

 current = self.head

 values = []

 while current:

 values.append(str(current.data))

 current = current.next

 print(" → ".join(values))

linked_list = LinkedList()

linked_list.insert(10)

linked_list.insert(20)

linked_list.insert(30)

linked_list.display()

```

**Explanation:** Each node stores data and a pointer to the next element. The `display` method performs a full traversal from the head node to the tail.

## Task 4: Binary Search Tree (BST)

**Question:** Use AI to create a BST with insert and in-order traversal methods.

Prompt used for AI: Generate a Python Binary Search Tree with a Node class and BST class. Implement recursive insert and in-order traversal. Add docstrings and demonstrate the sorted traversal output.

**Answer / Code:**

```
class BSTNode:
```

```
 """A node in a binary search tree."""
```

```
 def __init__(self, key):
```

```
 self.key = key
```

```
 self.left = None
```

```
 self.right = None
```

```
class BST:
```

```
 """Binary Search Tree supporting recursive insertion and in-order traversal."""
```

```
 def __init__(self):
```

```
 self.root = None
```

```
 def insert(self, key):
```

```
 """Insert a key into the BST."""
```

```
 self.root = self._insert_recursive(self.root, key)
```

```
 def _insert_recursive(self, node, key):
```

```
 if node is None:
```

```

 return BSTNode(key)

 if key < node.key:

 node.left = self._insert_recursive(node.left, key)

 elif key > node.key:

 node.right = self._insert_recursive(node.right, key)

 return node

def inorder(self):

 """Return the keys using in-order traversal."""

 result = []

 def traverse(node):

 if node:

 traverse(node.left)

 result.append(node.key)

 traverse(node.right)

 traverse(self.root)

 return result

tree = BST()

for value in [50, 30, 70, 20, 40, 60, 80]:

 tree.insert(value)

print("In-order:", tree.inorder())

```

**Explanation:** In-order traversal (Left, Root, Right) ensures that nodes in a Binary Search Tree are visited in non-decreasing order.

## Task 5: Hash Table

Question: Use AI to implement a hash table with basic `insert`, `search`, and `delete` methods.

**Prompt used for AI:**Generate a Python hash table with insert, search, and delete methods. Use chaining to handle collisions. Implement a simple hash function, add docstrings, and demonstrate insertion, searching, and deletion.

**Answer / Code:**`class HashTable:`

```
 """A simple hash table using chaining for collision handling."""
```

```
 def __init__(self, size=10):
```

```
 self.size = size
```

```
 self.table = [[] for _ in range(size)]
```

```
 def _hash(self, key):
```

```
 return hash(key) % self.size
```

```
 def insert(self, key, value):
```

```
 """Insert or update a key-value pair."""
```

```
 index = self._hash(key)
```

```
 for pair in self.table[index]:
```

```
 if pair[0] == key:
```

```
 pair[1] = value
```

```
 return
```

```

 self.table[index].append([key, value])

def search(self, key):

 """Return the value for a key, or None if it is not present."""

 index = self._hash(key)

 for pair in self.table[index]:

 if pair[0] == key:

 return pair[1]

 return None

def delete(self, key):

 """Delete a key and return True if it existed."""

 index = self._hash(key)

 for i, pair in enumerate(self.table[index]):

 if pair[0] == key:

 self.table[index].pop(i)

 return True

 return False

table = HashTable()

table.insert("CS101", "Python")

table.insert("CS102", "Data Structures")

print(table.search("CS101"))

```

```
print(table.delete("CS101"))
```

```
print(table.search("CS101"))
```

**Explanation:** Chaining is used to resolve collisions by storing multiple entries in a list at each hash index (bucket).

## Task 6: Graph (Adjacency List)

**Question:** Use AI to implement a graph using an adjacency list.

**Prompt used for AI:** Generate a Python Graph class using an adjacency list. Include methods to add vertices, add edges, and display connections. Use an undirected graph and provide a small demonstration.

Answer / Code:

```
class Graph:
 """Undirected graph represented with an adjacency list."""
 def init(self):
 self.adjacency_list = {}
 def add_vertex(self, vertex):
 """Add a vertex if it does not already exist."""
 if vertex not in self.adjacency_list:
 self.adjacency_list[vertex] = []
 def add_edge(self, vertex1, vertex2):
 """Add an undirected edge between two vertices."""
 self.add_vertex(vertex1)
 self.add_vertex(vertex2)
 if vertex2 not in self.adjacency_list[vertex1]:
 self.adjacency_list[vertex1].append(vertex2)
```

```
if vertex1 not in self.adjacency_list[vertex2]:
 self.adjacency_list[vertex2].append(vertex1)
def display(self):
```

```
 """Display each vertex and its connected vertices."""
```

```
 for vertex, neighbors in self.adjacency_list.items():
```

```
 print(f'{vertex}: {", ".join(map(str, neighbors))}')
 """
```

```
graph = Graph()
graph.add_edge("A", "B")
graph.add_edge("A", "C")
graph.add_edge("B", "D")
graph.display()
```

**\*\*Explanation:\*\*** The adjacency list representation uses a dictionary where keys are vertices and values are lists of neighbors, providing an efficient way to represent sparse graphs.

### ## Task 7: Priority Queue

**\*\*Question:\*\*** Use AI to implement a priority queue using Python's `heapq` module.

**\*\*Prompt used for AI:\*\***

> Generate a Python priority queue using heapq. Implement enqueue with a priority, dequeue, and display. Define clearly how priority is interpreted, add docstrings, and demonstrate the operations.

**\*\*Answer / Code:\*\***

```
```python
```

```
import heapq
```

```
class PriorityQueue:
```

```
    """Priority queue where a smaller priority number is served first."""
```

```
def __init__(self):

    self.items = []

def enqueue(self, item, priority):

    """Add an item with its priority."""

    heapq.heappush(self.items, (priority, item))

def dequeue(self):

    """Remove and return the highest-priority item, or None if empty."""

    if not self.items:

        return None

    return heapq.heappop(self.items)

def display(self):

    """Display queue items in priority order."""

    print(sorted(self.items))

queue = PriorityQueue()

queue.enqueue("Emergency", 1)

queue.enqueue("Normal", 3)

queue.enqueue("Important", 2)

queue.display()

print("Served:", queue.dequeue())
```

Explanation: Python's `heapq` module implements a min-heap by default, meaning the entry with the lowest numerical priority value is dequeued first.

Task 8: Double-Ended Queue (Deque)

Question: Use AI to implement a double-ended queue using `collections.deque`.

Prompt used for AI: Generate a Python `DequeDS` class using `collections.deque`. Include insert and remove operations at both ends, add docstrings, and demonstrate the operations.

Answer / Code: `from collections import deque`

```
class DequeDS:
```

```
    """Double-ended queue supporting insertion and removal at both ends."""
```

```
    def __init__(self):
```

```
        self.items = deque()
```

```
    def insert_front(self, item):
```

```
        """Insert an item at the front."""
```

```
        self.items.appendleft(item)
```

```
    def insert_rear(self, item):
```

```
        """Insert an item at the rear."""
```

```
        self.items.append(item)
```

```
    def remove_front(self):
```

```
        """Remove and return the front item, or None if empty."""
```

```
        if not self.items:
```

```

        return None

    return self.items.popleft()

def remove_rear(self):

    """Remove and return the rear item, or None if empty."""

    if not self.items:

        return None

    return self.items.pop()

def display(self):

    """Display the deque contents."""

    print(list(self.items))

d = DequeDS()

d.insert_rear(10)

d.insert_front(5)

d.insert_rear(20)

d.display()

print("Front removed:", d.remove_front())

print("Rear removed:", d.remove_rear())

```

Explanation: `collections.deque` is optimized for $O(1)$ appends and pops from either side, making it highly efficient for double-ended operations.

Task 9: Real-Time Application Challenge

Scenario: Designing a Campus Resource Management System.

Feature	Data Structure	Justification
Student Attendance Tracking	Hash Table ▾	High-speed lookup by Student ID for recording and retrieving presence.
Event Registration	Hash Table ▾	Facilitates rapid retrieval and modification of attendee details by ID.
Library Book Borrowing	Priority Queue ▾	Automates processing of returns or overdue alerts based on due dates.
Bus Scheduling	Graph ▾	Represents complex relationships between stops and route paths effectively.
Cafeteria Order Queue	Queue ▾	Ensures a fair, chronological service model for students placing orders.

Selected Feature Implementation (Cafeteria Order Queue):

```
from collections
import deque

class CafeteriaQueue:

    """Manage cafeteria orders using FIFO order."""

    def __init__(self):

        self.orders = deque()
```

```

def add_order(self, student, item):

    """Add a new cafeteria order to the rear."""

    self.orders.append((student, item))

def serve_order(self):

    """Serve the oldest order, or return None if no orders exist."""

    if not self.orders:

        return None

    return self.orders.popleft()

def display_orders(self):

    """Display pending orders."""

    for student, item in self.orders:

        print(f"{student}: {item}")

cafeteria = CafeteriaQueue()

cafeteria.add_order("Asha", "Idli")

cafeteria.add_order("Rahul", "Dosa")

cafeteria.display_orders()

print("Serving:", cafeteria.serve_order())

```

Task 10: Smart E-Commerce Platform

Scenario: Selection of structures for an Online Shopping System.

Feature	Data Structure	Justification
Shopping Cart Management	Linked List	Ideal for dynamic additions and removals during a shopping session.
Order Processing	Queue	Processes customer purchases in the exact order they were finalized.
Top-Selling Tracker	Priority Queue	Maintains a live ranking of products based on sales volume.
Product Search Engine	Hash Table	Enables near-instantaneous product detail retrieval using unique SKU keys.
Delivery Route Planning	Graph	Models delivery paths between warehouses and customer addresses.

Selected Feature Implementation (Product Search Engine):

```
class
ProductSearch:

    """Store and search products by product ID."""

    def __init__(self):

        self.products = {}

    def add_product(self, product_id, name, price):

        """Add or update a product using its ID."""

        self.products[product_id] = {"name": name, "price": price}
```

```

def search_product(self, product_id):

    """Return product details for an ID, or None if not found."""

    return self.products.get(product_id)

def delete_product(self, product_id):

    """Delete a product by ID."""

    return self.products.pop(product_id, None) is not None

catalog = ProductSearch()

catalog.add_product("P101", "Laptop", 55000)

print(catalog.search_product("P101"))

```

Veriffication & Conclusion

The implemented code modules were verified for logic and output consistency. Each data structure was chosen based on the computational complexity requirements of the specific task (e.g., $O(1)$ for hash lookups and $O(\log n)$ for priority heap operations). The resulting programs demonstrate robust handling of basic operations and error conditions.